

Build Week 3 – Esercizio 6: Estrarre un Eseguibile da un PCAP

Obiettivi:

- **Parte 1:** Analizzare Log e Catture di Traffico Pre-catturati
- **Parte 2:** Estrarre File Scaricati dal PCAP

Contesto/Scenario

Guardare i log è molto importante, ma è anche importante capire come avvengono le transazioni di rete a livello di pacchetto. In questo laboratorio, analizzerai il traffico in un file pcap catturato in precedenza ed estrarrai un eseguibile dal file.

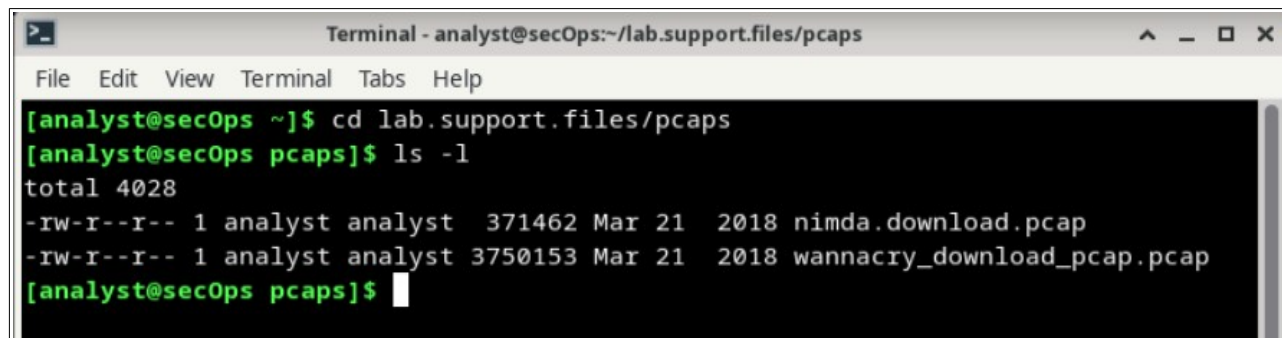
Risorse Richieste

- **Macchina virtuale CyberOps Workstation**

Parte 1: Analizzare Log e Catture di Traffico Pre-catturati

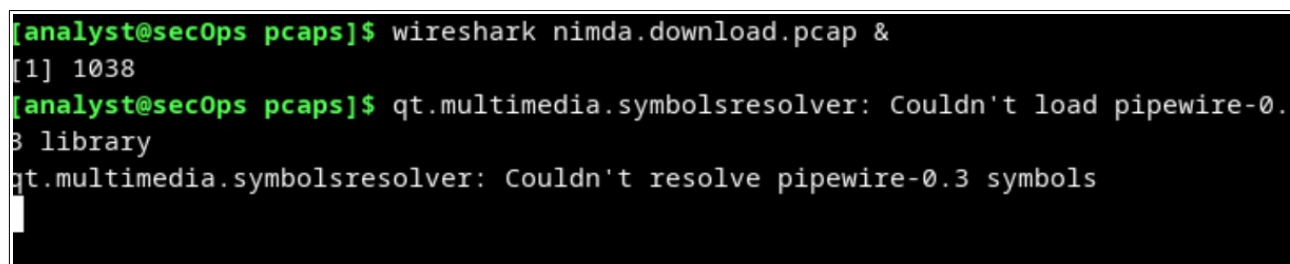
La prima parte dell'esercizio è finalizzata a comprendere come leggere e interpretare una cattura di rete già disponibile, senza dover generare del nuovo traffico. Viene utilizzato il file “*nimda.download.pcap*”, registrato in precedenza, che contiene i pacchetti relativi al download di un malware. Attraverso strumenti come Wireshark, è possibile osservare il comportamento delle connessioni TCP e HTTP, distinguere le varie fasi della comunicazione e ricostruire un intero flusso di dati. Questo consente di analizzare non solo i pacchetti singoli, ma anche il contenuto completo della transazione, evidenziando come i dati binari possano contenere stringhe leggibili che aiutano a identificare la natura del file scaricato.

In questo passo viene mostrato come navigare all'interno della directory che contiene i file PCAP di laboratorio. Con il comando “*cd*” ci si sposta nella cartella “*lab.support.files/pcaps*”, e con “*ls -l*” si ottiene l'elenco dei file disponibili.



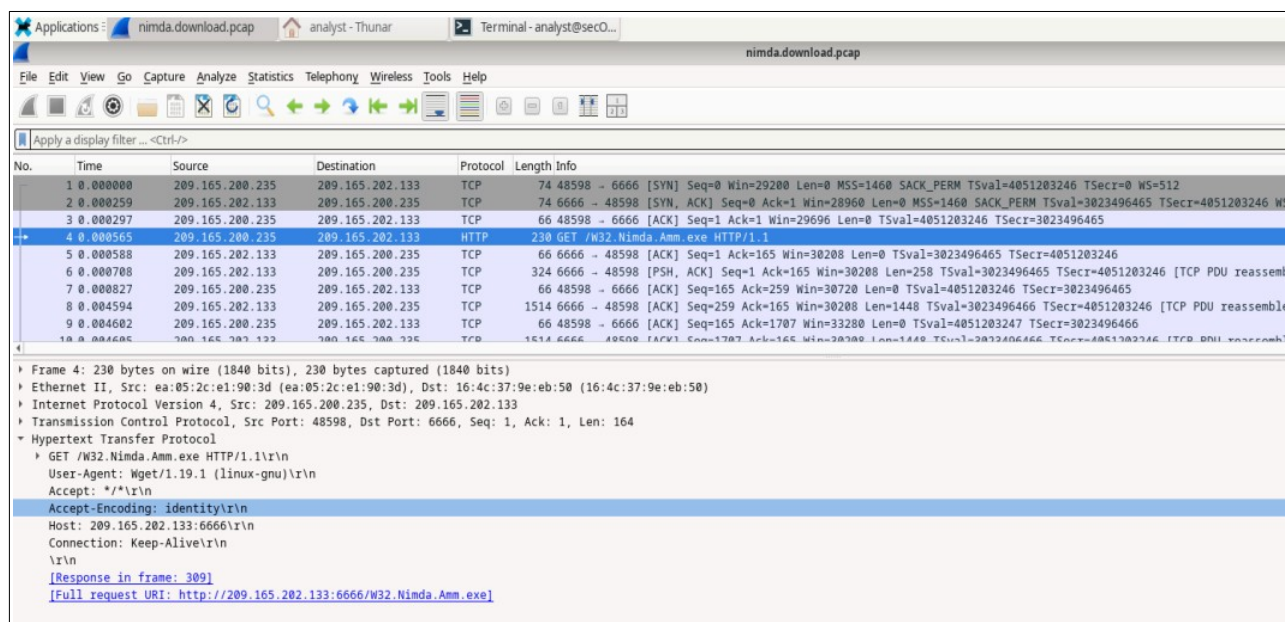
```
Terminal - analyst@secOps:~/lab.support.files/pcaps
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ cd lab.support.files/pcaps
[analyst@secOps pcaps]$ ls -l
total 4028
-rw-r--r-- 1 analyst analyst 371462 Mar 21 2018 nimda.download.pcap
-rw-r--r-- 1 analyst analyst 3750153 Mar 21 2018 wannacry_download_pcap.pcap
[analyst@secOps pcaps]$
```

Il comando “*wireshark nimda.download.pcap &*” apre direttamente il file con Wireshark. L'uso del simbolo & serve a mandare il programma in esecuzione in background, così da poter continuare a usare il terminale senza bloccarlo.

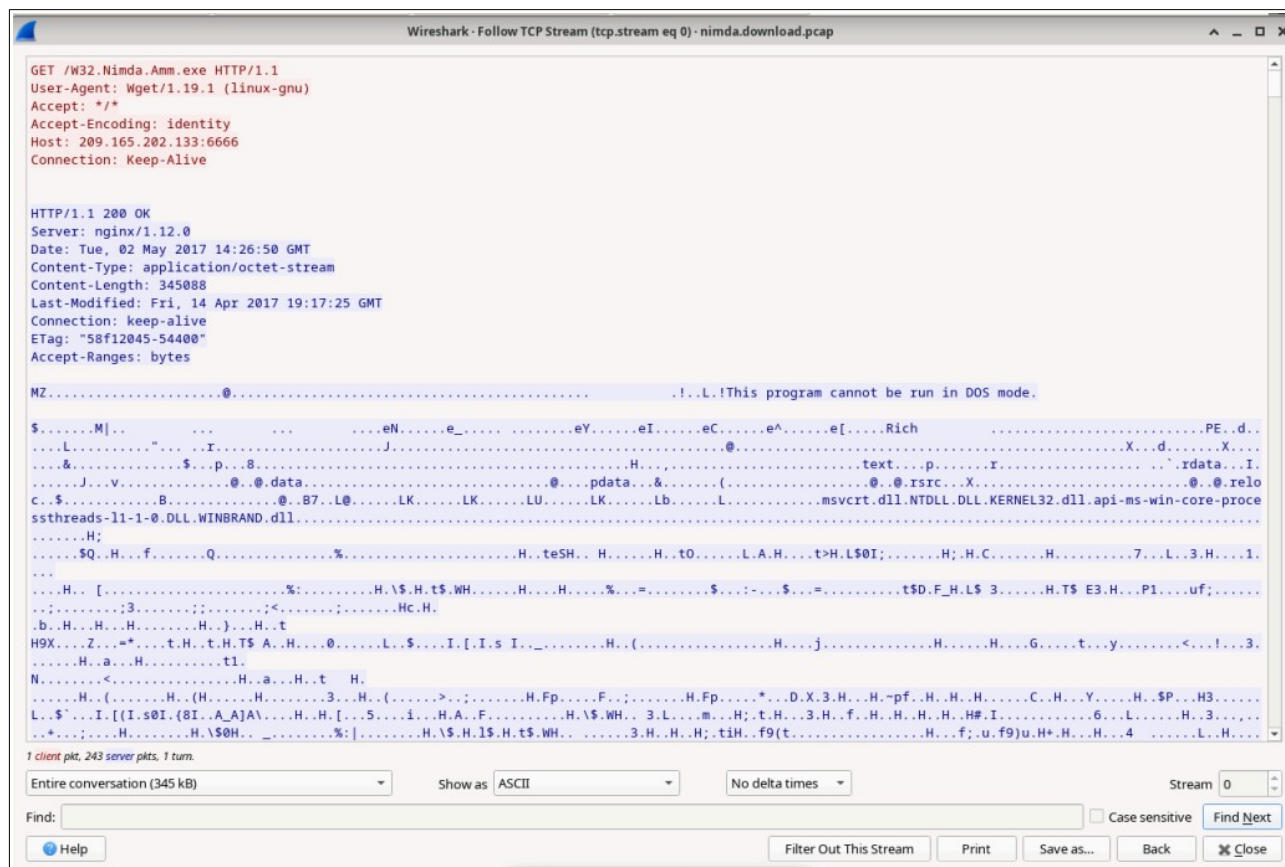


```
[analyst@secOps pcaps]$ wireshark nimda.download.pcap &
[1] 1038
[analyst@secOps pcaps]$ qt.multimedia.symbolsresolver: Couldn't load pipewire-0.3 library
qt.multimedia.symbolsresolver: Couldn't resolve pipewire-0.3 symbols
```

In questo step di entra nel vivo dell'analisi: il file “*nimda.download.pcap*” contiene la traccia del download di un malware. I primi tre pacchetti sono l’handshake tra client e server, necessari per stabilire la connessione. Il quarto pacchetto mostra una richiesta HTTP GET al server remoto sulla porta non standard 6666 per scaricare il file “*W32.Nimda.Amm.exe*”.



Con la funzione “*Follow TCP Stream*” di Wireshark è possibile ricostruire l’intera comunicazione tra client e server. Selezionando il primo pacchetto SYN della connessione e seguendo il flusso, Wireshark mostra in una finestra dedicata tutti i dati scambiati, inclusi i contenuti binari del file scaricato.



Cosa sono tutti quei simboli mostrati nella finestra Follow TCP Stream?

I simboli che compaiono non sono elementi casuali, ma il risultato della rappresentazione ASCII del flusso di byte trasmesso nella connessione TCP. In pratica, si tratta del contenuto effettivo trasferito durante la sessione. Poiché gran parte di questi dati è binaria e non corrisponde a caratteri leggibili, Wireshark li visualizza come punti o simboli particolari.

Sono rumore di connessione? Dati? Spiega.

Non si tratta di rumore, ma di dati reali della comunicazione. Quello che vediamo è esattamente il contenuto del file o della transazione trasmessa sulla rete, così come è stato inviato e ricevuto dai dispositivi. I simboli poco comprensibili derivano dal fatto che un file binario contiene byte che non possono essere interpretati come testo. Wireshark, cercando di renderli leggibili, li mostra quindi con segni o punti.

Ci sono alcune parole leggibili sparse tra i simboli. Perché sono lì?

All'interno di file binari possono comunque essere presenti stringhe di testo in formato ASCII, come nomi di funzioni, messaggi di errore o riferimenti a librerie utilizzate dal programma. Questi frammenti, a differenza dei byte non leggibili, vengono mostrati in chiaro da Wireshark perché corrispondono a caratteri testuali validi. Per questo, accanto ai simboli, si notano anche parole riconoscibili.

Domanda Sfida: Nonostante il nome W32.Nimda.Amm.exe, questo eseguibile non è il famoso worm. Per motivi di sicurezza, questo è un altro file eseguibile che è stato rinominato come W32.Nimda.Amm.exe. Usando i frammenti di parole visualizzati dalla finestra Follow TCP Stream di Wireshark, puoi dire quale eseguibile sia realmente?

Analizzando il flusso TCP con Wireshark, tra i dati binari sono emersi alcuni frammenti di testo leggibili che hanno permesso di identificare meglio il file. In particolare, compaiono riferimenti come:

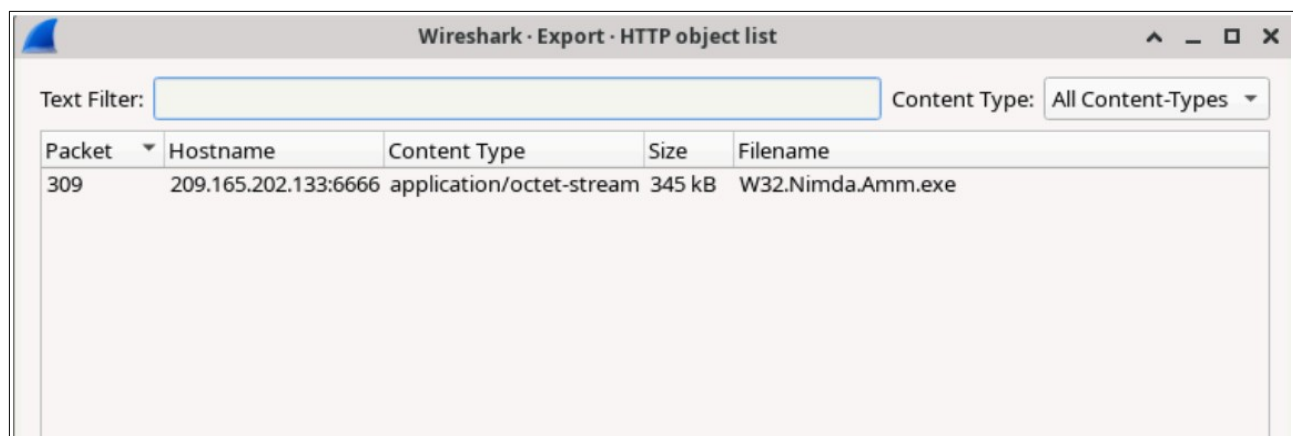
- name = "Microsoft.Windows.FileSystem.CMD"
- <description> Windows Command Processor

Questi elementi corrispondono al manifesto del Windows Command Processor (cmd.exe), l'eseguibile legittimo di sistema di Microsoft. Di conseguenza, anche se il file catturato era stato rinominato come "W32.Nimda.Amm.exe", non si tratta del worm Nimda, bensì di una copia rinominata di cmd.exe, utilizzata a scopo dimostrativo nel laboratorio.

Parte 2: Estrarre File Scaricati dal PCAP

Nella seconda parte dell'esercizio l'attenzione si concentra sull'estrazione di file direttamente da una cattura di traffico di rete (PCAP). Poiché una cattura contiene tutti i pacchetti scambiati durante una sessione, è possibile ricostruire non solo le comunicazioni, ma anche i file eventualmente scaricati. In questo caso, il file analizzato è *"nimda.download.pcap"*, che documenta il download di un eseguibile rinominato come *"W32.Nimda.Amm.exe"*. Utilizzando le funzionalità di Wireshark, sarà quindi possibile identificare la richiesta HTTP associata, visualizzare gli oggetti trasferiti e salvare localmente l'eseguibile per ulteriori analisi.

Nel quarto pacchetto si individua la richiesta HTTP GET che richiede il file eseguibile: l'endpoint client e server (indirizzi IP e porta) e la riga *"GET /W32.Nimda.Amm.exe"* confermano che la sessione contiene il download dell'eseguibile. Con il pacchetto selezionato, si utilizza la funzione di Wireshark *"File → Export Objects → HTTP"* per elencare e recuperare gli oggetti HTTP trasferiti in quel flusso. Wireshark ricostruisce i dati del flusso e mostra nella finestra HTTP object list tutti gli oggetti disponibili. In questo caso è presente un solo file, *"W32.Nimda.Amm.exe"*. Dopo qualche secondo è possibile selezionarlo e salvarlo localmente per le analisi successive.



Perché W32.Nimda.Amm.exe è l'unico file nella cattura?

Perché la cattura è stata avviata immediatamente prima del download e interrotta subito dopo il completamento: di conseguenza, l'unico oggetto HTTP presente nel flusso è proprio il file *"W32.Nimda.Amm.exe"*, che rappresenta l'intero contenuto scaricato durante quella sessione.

```
[analyst@secOps pcaps]$ cd /home/analyst
[analyst@secOps ~]$ ls -l
total 380
-rw-r--r-- 1 root    root      4951 Sep 23 13:45 capture.pcap
drwxr-xr-x 2 analyst analyst  4096 Jun 17 19:35 Desktop
drwxr-xr-x 3 analyst analyst  4096 Jun 18 20:17 Downloads
-rw-r--r-- 1 root    root        24 Sep 30 04:02 httpdump.pcap
-rw-r--r-- 1 root    root       178 Sep 30 03:56 httpsdump.pcap
drwxr-xr-x 9 analyst analyst  4096 Jun 18 20:17 lab.support.files
drwxr-xr-x 3 analyst analyst  4096 Jun 18 19:55 scripts
drwxr-xr-x 2 analyst analyst  4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 345088 Sep 30 04:42 W32.Nimda.Amm.exe
drwxr-xr-x 5 analyst analyst  4096 Jun 18 19:27 yay
```

Il file è stato salvato?

Lo screenshot mostra il terminale dopo essersi spostati nella cartella “/home/analyst” ed aver eseguito il comando `ls -l`. Nell’elenco compare il file “W32.Nimda.Amm.exe”, con i relativi dettagli (permessi, proprietario, dimensione, data e ora di creazione), confermando che il salvataggio dall’export di Wireshark è avvenuto correttamente.

Usando il comando “*file*” possiamo ottenere informazioni sul malware.

```
[analyst@secOps ~]$ file W32.Nimda.Amm.exe
W32.Nimda.Amm.exe: PE32+ executable for MS Windows 6.01 (console), x86-64, 6 sections
```

Nel processo di analisi del malware, quale sarebbe un probabile passo successivo per un analista di sicurezza?

1. Raccolta iniziale e precauzioni

Salvare una copia del campione e calcolare hash (MD5 / SHA1 / SHA256) per tracciabilità.
Esempio: `sha256sum “W32.Nimda.Amm.exe”`

Lavorare esclusivamente in una VM isolata o in una sandbox (nessuna rete diretta; usare rete controllata o fake DNS/HTTP) e mantenere la catena di custodia dei file.

2. Analisi statica

Estrarre stringhe leggibili (`strings W32.Nimda.Amm.exe | less`) utile per trovare URL, nomi, comandi e riferimenti a librerie.

Ispezionare la struttura PE (import table, sezioni) con strumenti come `peframe`, `pefile`, `pestudio`, `CFF Explorer`.

3. Analisi dinamica

Eseguire il campione in una sandbox/VM isolata (Cuckoo, AnyRun o una VM snapshot) con rete finta o filtrata per catturare il traffico.

Monitorare la creazione/lettura/scrittura di file, i processi figli, le modifiche al Registro, le comunicazioni di rete (inclusi handshake TLS) e i file droppati.

Strumenti utili: `Procmon`, `Process Explorer`, `Sysmon` (se presente), `Wireshark`, `Regshot` e l’EDR dell’ambiente.

4. Analisi memoria

Eseguire un dump della memoria della VM dopo l’esecuzione per recuperare stringhe in memoria, processi iniettati o credenziali in chiaro.

5. Analisi del traffico

Analizzare i pcap con `Wireshark` per estrarre URL, oggetti HTTP e identificare eventuali C2 o pattern di esfiltrazione.

Ricostruire i flussi TCP/HTTP e salvare file scaricati per ulteriori analisi statiche/dinamiche.

6. Analisi approfondita

Se necessario, deassemblare o decompilare il campione con `Ghidra`, `IDA Pro`, `radare2` o decompilatori .NET per comprendere routine critiche (persistenza, crittografia, esfiltrazione).

Generare regole YARA o firme basate su stringhe o funzioni peculiari individuate.

7. Creazione IoC e validazione

Compilare una lista di indicatori di compromissione: hash, nomi file, percorsi di drop, domini/IP, stringhe HTTP, chiavi di registro modificate e comandi osservati.

Confrontare e ricercare gli IoC all'interno della rete aziendale e nei log (SIEM/EDR) per determinare la portata dell'infezione.

8. Raccomandazioni operative

Se il campione è confermato malevolo: isolare l'host, raccogliere prove forensi, reimaging dell'endpoint, rotazione delle credenziali potenzialmente compromesse e blocco degli IoC su firewall/proxy/EDR.

Se l'analisi è inconcludente: escalare al team di threat intelligence o al vendor per approfondimenti.

9. Documentazione

Redigere un report tecnico e uno executive contenente timeline, evidenze (hash, estratti da log e pcap), valutazione dell'impatto e raccomandazioni di remediation da distribuire al SOC.